

Foundations of Information Engineering & Cryptography Notes

Cointreau

October 7, 2026

Contents

1	Unit 1 — Introduction to Coding and Cryptography	10
1.1	Short Introduction	10
1.1.1	Digital Communication System	10
1.1.2	Bit Errors and Noise	10
1.1.3	Two Basic Strategies for Bit Errors	11
1.2	Basics of Cryptography	11
1.2.1	Cryptology, Cryptography and Cryptanalysis	11
1.2.2	Four Main Goals of Cryptography	11
1.2.3	Basic Language of Cryptography	11
1.2.4	Key Space and Key Length	12
1.2.5	Classical Ciphers	12
1.2.6	Caesar Cipher	12
1.2.7	Monoalphabetic Substitution and Frequency Analysis	13
1.2.8	Polyalphabetic Substitution and Vigenère Cipher	13
1.2.9	Columnar Transposition	14
1.2.10	Kerckhoffs's Principle and Security by Obscurity	15
1.3	Modulo Arithmetic for Cryptography	15
1.3.1	Modulo Representation	15
1.3.2	Congruence Preserved by Arithmetic	15

1.3.3	A Divisibility Rule from Modulo Arithmetic	16
1.4	Basics of Error Detection and Correction	16
1.4.1	HKID Example	16
1.4.2	Single Parity Check	17
1.4.3	Two-Dimensional Parity Check	17
1.4.4	Repetition Code	17
1.4.5	(n, k) Binary Codes	17
1.4.6	A Non-Systematic Example	18
1.4.7	Code Rate	18
1.4.8	Hamming Code	18
2	Unit 2 — Functions	20
2.1	Opening Idea: Different Sizes of Infinity	20
2.2	Sets	20
2.2.1	Ordered Pairs	20
2.2.2	Cartesian Product	21
2.2.3	Power Set	21
2.2.4	Partitions	21
2.3	Functions	21
2.3.1	Inverse Image	22
2.3.2	Composition of Functions	22
2.3.3	Composition Example	23
2.4	Injections and Surjections	23
2.4.1	Identity Function	23
2.4.2	Inverse Function	23
2.4.3	Important Counterexample	24
2.4.4	A Floor-Function Example	24
2.4.5	Modulo-10 Function	25
2.5	Countable and Uncountable Sets	25
2.5.1	Countable Examples	25

2.5.2	Union of Countable Sets	25
2.5.3	Uncountability of an Interval	26
2.6	Hash Functions	26
2.6.1	Overflow	26
2.6.2	Why a Fixed-Length Hash Cannot Be an Injection Here	26
3	Unit 3 — Relations	27
3.1	Definition of Relations	27
3.1.1	Relation versus Function	27
3.1.2	Inverse Relation	27
3.1.3	Composition of Relations	27
3.1.4	k -ary Relations	28
3.1.5	Directed Graph Representation	28
3.2	Properties of Relations	28
3.2.1	Congruence Modulo 3	28
3.2.2	Divisibility as a Relation	29
3.3	Equivalence Relations	29
3.3.1	Motivation	29
3.3.2	Partition-Induced Relation	29
3.3.3	Test 1 Partition Example	29
3.4	Equivalence Classes	29
3.4.1	Proof of the Disjoint-or-Equal Property	30
3.4.2	Equivalence Class Example	30
3.5	More Examples of Equivalence Relations	30
3.5.1	Binary Sequences by Parity of Number of 1s	30
3.5.2	Logic Circuit Equivalence	31
3.5.3	Calendar Modulo 7	31
3.5.4	Pure Mathematics Example	31
3.5.5	Relation Example	31

4	Unit 4 — Divisibility and Euclidean Algorithm	32
4.1	Number Theory and Cryptography	32
4.2	Divisibility	32
4.2.1	Divisibility as a Relation	32
4.2.2	Quotient-Remainder Theorem	33
4.3	Primes and Co-primes	33
4.3.1	Infinitely Many Primes	34
4.3.2	Greatest Common Divisor	34
4.3.3	Euler's Totient Function	35
4.4	Euclidean Algorithm	36
4.4.1	Three Ways to Compute a GCD	36
4.4.2	Euclidean Algorithm	37
4.5	Extended Euclidean Algorithm	38
4.6	Fundamental Theorem of Arithmetic	39
4.6.1	Euclid's Lemma	40
5	Unit 5 — Modular Arithmetic and Chinese Remainder Theorem	41
5.1	What is Modular Arithmetic?	41
5.1.1	Field Idea	41
5.1.2	Arithmetic under Congruence	41
5.2	Diophantine Equations	41
5.3	Modular Division	42
5.3.1	Why division is different	42
5.3.2	Division Modulo a Prime	43
5.4	Modular Exponentiation	43
5.4.1	Fermat's Little Theorem	44
5.4.2	Euler's Theorem	44
5.5	Chinese Remainder Theorem	45
5.5.1	Motivating Examples	45
5.5.2	Sunzi's 3-5-7 Problem	45

5.5.3	Why CRT Matters Later	46
5.6	SageMath	46
6	Unit 6 — Public Key Cryptography	47
6.1	Cybersecurity Ethics	47
6.1.1	Codes of Ethics	47
6.1.2	(ISC) ² Code of Ethics	47
6.1.3	Cybersecurity Example	47
6.1.4	Types of Hackers	47
6.2	Public Key Cryptography	48
6.2.1	Two Problems Motivating Public-Key Cryptography	48
6.2.2	Two Main Motivations	48
6.2.3	Public-Key Requirements	48
6.2.4	Trapdoor One-Way Functions	48
6.2.5	Three Families Mentioned	48
6.2.6	Why Public-Key Systems Are Usually Used to Set Up a Session Key . .	48
6.2.7	HTTPS and SSL/TLS Workflow	49
6.2.8	Digital Signatures	49
6.2.9	Hash-Then-Sign	49
6.3	RSA Cryptosystems	50
6.3.1	Background	50
6.3.2	Preparing the Message	50
6.3.3	RSA Key Generation	50
6.3.4	RSA Encryption and Decryption	50
6.3.5	Why RSA Works	51
6.3.6	Why Factoring Matters for RSA Security	51
6.4	Diffie-Hellman Key Exchange	51
6.4.1	Discrete Logarithm	51
6.4.2	$GF(p)$, Order and Primitive Elements	52
6.4.3	Diffie-Hellman Protocol	52

6.4.4	Why a Passive Eve Has Difficulty	53
6.4.5	Man-in-the-Middle Attack	53
6.5	El Gamal Cryptosystems	53
6.5.1	Key Generation	53
6.5.2	Encryption	53
6.5.3	Decryption	54
7	Unit 7 — Digital Certificate	55
7.1	Motivation: Identity Verification Problem	55
7.2	Public-Key Authority	55
7.3	Digital Certificates	55
7.4	Issuing a Digital Certificate	55
7.5	Using a Digital Certificate	56
7.6	X.509	56
7.6.1	X.509 Certificate Structure	56
7.7	Self-Signed Certificate	57
7.8	OpenSSL Practical Activity	57
7.9	TBL Preparation	57
8	Unit 8 — Linear Algebra for Cryptography	58
8.1	Opening Idea: Four-Digit Combination Lock	58
8.2	Linear Mappings	58
8.2.1	Vectors	58
8.2.2	Vector-Valued Mappings	58
8.2.3	Linearity	58
8.2.4	Zero-in, Zero-out	59
8.2.5	Examples and Non-Examples	59
8.2.6	Inner Product	59
8.2.7	A Linear Function from an Inner Product	59
8.3	Matrix Representation of Linearity	60

8.3.1	Matrix-Vector Multiplication	60
8.3.2	Every Matrix Defines a Linear Map	60
8.3.3	Geometric Transformations	60
8.3.4	Converse Theorem	61
8.3.5	Affine Functions	61
8.3.6	Composition of Linear Maps	61
8.4	Determinants and Matrix Inverses	61
8.4.1	Determinant of a 2×2 Matrix	61
8.4.2	Minor and Cofactor	61
8.4.3	Cofactor Expansion	62
8.4.4	Cramer's Rule	62
8.4.5	Matrix Inverse by Cofactors	62
8.5	Symmetric Ciphers	63
8.5.1	Affine Cipher	63
8.5.2	Hill Cipher	63
8.5.3	Invertibility Modulo 26	63
8.5.4	Known-Plaintext Vulnerability	64
8.5.5	Caesar with an Increasing Shift	64
8.5.6	Optional Enigma Material	64
8.6	Secret Sharing	64
8.6.1	Two-Point Idea	64
8.6.2	(t, n) Threshold Secret Sharing	65
8.6.3	Polynomial Secret Sharing	65
8.6.4	Recovery as a Linear Algebra Problem	65
9	Unit 9 — Error-Correcting Codes	66
9.1	Introduction	66
9.1.1	(n, k) Binary Codes	66
9.1.2	Single Parity Check Revisited	66
9.1.3	Code Rate	66

9.1.4	Linear Codes	66
9.1.5	Matrix-Matrix Multiplication	67
9.2	Error Detection Capability	67
9.2.1	Why Single Parity Can Fail	67
9.2.2	Hamming Distance is a Metric	67
9.2.3	Minimum Distance	67
9.3	Error Correction Capability	68
9.3.1	(5,1) Repetition Code	68
9.3.2	Repetition with an Extra Parity Bit	69
9.4	Generator Matrices for Encoding	69
9.4.1	Injectivity of Encoding	69
9.4.2	Linear Codes as Vector Subspaces	70
9.4.3	Minimum Distance of a Linear Code	70
9.5	Parity-Check Matrices for Decoding	70
9.5.1	Parity-Check Equation	70
9.5.2	Simple Parity Example	70
9.5.3	Systematic Form	70
9.5.4	Generator Matrix of the (5, 2) Example	71
9.5.5	Syndrome Example	71
9.6	More on Hamming Codes	71
9.6.1	The (7, 4) Hamming Code	71
9.6.2	Hamming Code Capability	72
9.6.3	Parity-Check Matrix	72
9.6.4	Single-Bit Syndrome Decoding	72
9.6.5	Unit 1 / Test 1 Hamming Example Revisited	73
9.6.6	Error Detection versus Correction Trade-off	73
10	Cross-Unit Exam Workflow	74
10.1	Proving a Function is Injective	74
10.2	Proving a Function is Surjective	74

10.3 Proving an Equivalence Relation	74
10.4 Finding an Equivalence Class	75
10.5 Extended Euclidean Algorithm	75
10.6 Finding a Modular Inverse	75
10.7 Modular Exponentiation	75
10.8 RSA	76
10.9 Diffie-Hellman	76
10.10CRT	76
10.11Error-Correcting Codes	76
10.12Exam-Style Coverage	77
11 Quick Formula and Definition Reference	78
11.1 Functions	78
11.2 Relations	78
11.3 Divisibility and GCD	78
11.4 Modular Arithmetic	78
11.5 RSA	78
11.6 Diffie-Hellman	79
11.7 El Gamal	79
11.8 Linear Algebra	79
11.9 Error-Correcting Codes	79
12 Source Coverage Map	80

1 Unit 1 — Introduction to Coding and Cryptography

1.1 Short Introduction

1.1.1 Digital Communication System

The models digital communication as a sequence of operations:

Data Source $\rightarrow$ Source Encoder $\rightarrow$ Encrypter $\rightarrow$ Channel Encoder $\rightarrow$ Digital Modulator
 $\rightarrow$ Channel $\rightarrow$ Digital Demodulator $\rightarrow$ Channel Decoder $\rightarrow$ Decrypter $\rightarrow$ Source Decoder $\rightarrow$ Sink.

- The source may contain bits, digits, or English characters.
- Source encoding can perform compression.
- Encryption provides secrecy, while channel encoding introduces redundancy for error handling.

1.1.2 Bit Errors and Noise

Suppose N bits are transmitted. Because of channel noise, the received bit vector may differ from the transmitted bit vector.

Definition A bit error occurs when a transmitted bit is received incorrectly. The probability that a bit error occurs is called the **bit error rate**.

Let

$$x = (x_1, x_2, \dots, x_N), \quad y = (y_1, y_2, \dots, y_N).$$

An error vector

$$e = (e_1, e_2, \dots, e_N),$$

where $e_i = 1$ when bit i is in error and $e_i = 0$ otherwise. Then

$$y_i = x_i + e_i,$$

where binary addition is logical XOR.

XOR table

A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	0

1.1.3 Two Basic Strategies for Bit Errors

1. **Error correction:** include enough redundant information to identify which bits are in error and correct them.
2. **Error detection:** include enough redundant information to detect that an error occurred. The receiver may then request retransmission.

1.2 Basics of Cryptography

1.2.1 Cryptology, Cryptography and Cryptanalysis

Definition

1. **Cryptology** is the science of making and breaking secret codes.
2. **Cryptography** is the development and use of codes and techniques for secure communications.
3. **Cryptanalysis** is breaking codes by exploiting weaknesses.

1.2.2 Four Main Goals of Cryptography

1. **Confidentiality:** only authorized users can read the message.
2. **Data integrity:** the message was not altered.
3. **Origin authentication:** the message is not a forgery and really comes from the stated sender.
4. **Non-repudiation:** the sender cannot later repudiate or refute the validity of the message sent.

1.2.3 Basic Language of Cryptography

Let M be the message space, K the key space, and C the ciphertext space. The encryption as

$$f : M \times K \rightarrow C, \quad c = f(m, k_A),$$

and decryption as

$$g : C \times K \rightarrow M, \quad m = g(c, k_B).$$

Symmetric cryptography Alice and Bob share a secret key k_S :

$$c = f(m, k_S), \quad m = g(c, k_S).$$

Public-key cryptography Bob has a public key k_B^+ and private key k_B^- :

$$c = f(m, k_B^+), \quad m = g(c, k_B^-).$$

An additional requirement is that knowing the public key should make it computationally hard to determine the private key.

1.2.4 Key Space and Key Length

The key space K is the set of all possible keys. Its size is $|K|$. It defines key length by

$$\text{key length} = \log_2 |K| \quad \text{bits.}$$

Adding one bit doubles the key space. Longer keys generally increase security but also increase resource usage and computational load.

1.2.5 Classical Ciphers

Classifies classical ciphers mainly as substitution or transposition ciphers.

Substitution The order of letters is preserved but symbols are replaced.

- Monoalphabetic substitution, including Caesar cipher.
- Polyalphabetic substitution, including the Vigenère cipher.

Transposition The same letters are rearranged into a different order. A columnar transposition is the example.

1.2.6 Caesar Cipher

Definition A Caesar cipher cyclically shifts every letter by a fixed number of positions. With $A = 0, \dots, Z = 25$,

$$f(m, k) = (m + k) \bmod 26,$$

and decryption is

$$g(c, k) = (c - k) \bmod 26.$$

For a non-zero shift, it gives a key space of size 25.

Example Encrypt HELLO with a Caesar shift of 3.

Step-by-Step

1. Use $A = 0, \dots, Z = 25$.
2. Shift each letter forward by 3.

Plaintext	<i>H</i>	<i>E</i>	<i>L</i>	<i>L</i>	<i>O</i>
Value	7	4	11	11	14
Shift	3	3	3	3	3
Cipher value	10	7	14	14	17
Ciphertext	<i>K</i>	<i>H</i>	<i>O</i>	<i>O</i>	<i>R</i>

3. Therefore the ciphertext is KHOOR.

1.2.7 Monoalphabetic Substitution and Frequency Analysis

A monoalphabetic cipher uses one fixed replacement mapping for the entire message. The example maps

$$abc \dots z \rightarrow mnbvcxzasdfghjklpoiuytrewq.$$

The key space is $26!$, approximately 4×10^{26} , but the cipher is still vulnerable to frequency analysis because letter-frequency patterns are preserved.

Definition A frequency attack exploits the fact that some letters occur more often than others in natural language. It specifically notes that *e* is the most common English letter.

1.2.8 Polyalphabetic Substitution and Vigenère Cipher

The Vigenère cipher as a method using multiple substitution alphabets so that frequency patterns are less directly exposed.

Step-by-Step Suppose the message is $m = (m_1, \dots, m_N)$ and the keyword is $k = (k_1, \dots, k_n)$, where $n \leq N$.

1. Repeat the keyword until it matches the message length.
2. Convert letters to the numerical range 0–25.
3. Encrypt each message letter using a Caesar shift specified by the corresponding repeated keyword letter.
4. Reduce modulo 26.

It gives key-space size 26^n .

Example Encrypt MOONCAKE using keyword BAY.

Step-by-Step

1. Repeat the keyword to the message length: BAYBAYBA.
2. Convert letters to numbers using $A = 0, \dots, Z = 25$.
3. Apply a Caesar shift using the corresponding keyword letter.

Position	1	2	3	4	5	6	7	8
Plaintext	M	O	O	N	C	A	K	E
Plain value	12	14	14	13	2	0	10	4
Key	B	A	Y	B	A	Y	B	A
Key value	1	0	24	1	0	24	1	0
Sum mod 26	13	14	12	14	2	24	11	4
Ciphertext	N	O	M	O	C	Y	L	E

4. Read the final row: NOMOCYLE.

Key Point The example shows Vigenère with greater resistance to frequency attacks than Caesar cipher.

1.2.9 Columnar Transposition

Step-by-Step

1. Write the plaintext row-by-row into a rectangular array whose row length is determined by the key length.
2. Add random null letters if necessary to fill the required matrix.
3. Determine the order of columns according to the alphabetical order of the key.
4. Read columns in that order.

Example The given uses key ZEBRAS. Its alphabetical column order is

6, 3, 2, 4, 1, 5.

For the message WE ARE DISCOVERED . FLEE AT ONCE, with the given null letters, the example gives ciphertext EVLNE ACDTK ESEAQ ROFOJ DEECU WIREE. The key space for a six-letter key is $6! = 240$.

1.2.10 Kerckhoffs's Principle and Security by Obscurity

Definition The Kerckhoffs principle states that Eve may know the cryptographic system, but not the secret key. Modern cryptography is based on the idea that the system need not be secret.

Why this is useful

- It is easier to keep a key secret than an entire system.
- It is easier to change a key than replace a whole system.
- Different users can use the same system with different secret keys.

1.3 Modulo Arithmetic for Cryptography

1.3.1 Modulo Representation

It represents the alphabet as

$$M = C = \{0, 1, \dots, 25\},$$

with $A = 0, B = 1, \dots, Z = 25$.

Definition Two integers a and b are congruent modulo n if they have the same remainder when divided by n :

$$a \equiv b \pmod{n}.$$

Example

$$28 \equiv 4 \pmod{12}, \quad 25 \equiv 33 \pmod{8}.$$

Step-by-Step Caesar encryption can therefore be expressed exactly as

$$f(m, k) = (m + k) \bmod 26.$$

For decryption,

$$g(c, k) = (c - k) \bmod 26.$$

1.3.2 Congruence Preserved by Arithmetic

If $a \equiv c \pmod{n}$ and $b \equiv d \pmod{n}$, then we states that congruence is preserved under addition, subtraction and multiplication:

$$a + b \equiv c + d \pmod{n},$$

$$a - b \equiv c - d \pmod{n},$$

$$ab \equiv cd \pmod{n}.$$

Step-by-Step Proof idea for addition If $a = kn + c$ and $b = hn + d$, then

$$a + b = (k + h)n + (c + d),$$

so $a + b$ and $c + d$ have the same remainder modulo n .

Example

$$10 \times 16 \times 17 + 25 \times 5 - 37 \equiv 1 \times 1 \times 2 + 1 \times 2 - 1 \equiv 0 \pmod{3}.$$

1.3.3 A Divisibility Rule from Modulo Arithmetic

The example proves the divisibility rule for 9.

Step-by-Step Write an n -digit number as

$$x = a_{n-1}10^{n-1} + a_{n-2}10^{n-2} + \cdots + a_110 + a_0.$$

Because $10 \equiv 1 \pmod{9}$, every $10^i \equiv 1 \pmod{9}$. Therefore,

$$x \equiv a_{n-1} + a_{n-2} + \cdots + a_1 + a_0 \pmod{9}.$$

So the number is divisible by 9 exactly when its digit sum is divisible by 9.

Example 4563 is divisible by 9 because $4 + 5 + 6 + 3 = 18$. The example contrasts this with 3145, whose digit sum is 13.

1.4 Basics of Error Detection and Correction

1.4.1 HKID Example

The example gives a Hong Kong ID example using weights

$$9, 8, 7, 6, 5, 4, 3, 2, 1$$

and a modulo-11 check. The example's alphabet mapping is $A = 10$, $B = 11$, and so on through $Z = 35$, with space represented by 36. The example reduces the weighted sum to

$$5 + 8 + 9 + 3 + 7 + 2 + 7 + 5 + x \equiv 0 \pmod{11},$$

so

$$x \equiv -2 \equiv 9 \pmod{11}.$$

1.4.2 Single Parity Check

Definition Add one parity bit to k information bits so that the total number of 1s is even (for even parity).

This creates a $(k + 1, k)$ code. It can detect a single-bit error, but the receiver does not know which bit is wrong.

Step-by-Step

1. Count the 1s in the information bits.
2. Add a parity bit that makes the total count even.
3. At the receiver, check whether the total number of 1s is still even.

1.4.3 Two-Dimensional Parity Check

We use a two-dimensional array of bits with row and column parity. It gives the activity of arranging 10101010101011 in a 4×4 array, computing parity bits, and checking the consistency of the final corner parity. The method is able to locate a single-bit error by identifying the row and column whose parity checks fail.

1.4.4 Repetition Code

Definition A repetition code repeats each bit a fixed number of times. With three repetitions,

$$0 \mapsto 000, \quad 1 \mapsto 111.$$

Decoding can be done by majority vote.

Example The example gives received word 010; majority is 0, so the decoded bit is 0.

1.4.5 (n, k) Binary Codes

A given (n, k) binary code contains:

- k information bits,
- r redundant/parity bits,
- $n = k + r$ coded bits.

The code C contains 2^k codewords in $\mathbb{B}^n$.

Definition An encoding is systematic if the information bits appear directly as part of the coded output; the parity bits do not have to come after the information bits.

Example The example discusses a (4, 3) even-parity code and a (5, 1) repetition code.

1.4.6 A Non-Systematic Example

The example gives

$$c_1 = u_1, \quad c_2 = u_1 + u_2 + u_3, \quad c_3 = u_2 + u_3, \quad c_4 = u_1 + u_2,$$

with binary addition. The code is non-systematic because the information bits u_2 and u_3 do not appear individually as coded positions.

1.4.7 Code Rate

Definition For an (n, k) code,

$$R_c = \frac{k}{n}.$$

It measures the proportion of transmitted bits that carry useful information. If a link transmits W bits/s, the effective information rate is $R_c W$ bits/s.

Example If 7 information bits are encoded with a single parity check, the code is (8, 7), so

$$R_c = \frac{7}{8}.$$

For a 1 Mbps link the useful information rate is therefore

$$\frac{7}{8}(1 \text{ Mbps}) = 0.875 \text{ Mbps}.$$

1.4.8 Hamming Code

It introduces Richard Hamming's famous (7, 4) code. It is designed for single-bit correction and double-bit-error detection capability.

The given encoding equations are

$$\begin{aligned} c_1 &= u_1, & c_2 &= u_2, & c_3 &= u_3, & c_4 &= u_4, \\ c_5 &= u_1 + u_2 + u_4, \\ c_6 &= u_1 + u_3 + u_4, \\ c_7 &= u_2 + u_3 + u_4. \end{aligned}$$

Its rate is

$$R_c = \frac{4}{7}.$$

Worked Example: Decoding Worked Example: received vector

$$y = (0, 1, 0, 1, 1, 0, 0),$$

and assumes at most one bit is in error. Using the three parity relationships represented by the Hamming-code Venn diagram, the example solution identifies the first bit as the erroneous bit and therefore recovers message

$$(u_1, u_2, u_3, u_4) = (1, 1, 0, 1).$$

Comparison with two-dimensional parity The comparison the $(7, 4)$ Hamming code with the two-dimensional parity-check code using a 2×2 information array. Both can correct a single-bit error in the described setup, but the Hamming code is more communication-efficient because its rate $4/7$ is greater than the two-dimensional parity code rate $4/9$.

2 Unit 2 — Functions

2.1 Opening Idea: Different Sizes of Infinity

It begins with the natural numbers

$$\{1, 2, 3, 4, \dots\}$$

and even numbers

$$\{2, 4, 6, 8, \dots\}$$

and asks which set is larger. The point is that the two sets have the same size in the sense developed later through bijections.

The material also introduces Cantor's idea that there are different infinities and Hilbert's Hotel as a thought experiment illustrating this idea.

2.2 Sets

Definition A set is a collection of objects. If x is an element of A , write $x \in A$; otherwise $x \notin A$.

Example

$$A = \{1, 3, 5, 7\}.$$

Then $1 \in A$ but $2 \notin A$.

Definition The empty set $\emptyset$ contains no elements.

Symbol	Name	Typical notation
$\mathbb{N}$	Natural numbers	$0, 1, 2, 3, \dots$ (some conventions omit 0)
$\mathbb{Z}$	Integers	$\dots, -2, -1, 0, 1, 2, \dots$
$\mathbb{B}$	Binary numbers	$\{0, 1\}$
$\mathbb{Q}$	Rational numbers	rationals
$\mathbb{R}$	Real numbers	reals
$\mathbb{C}$	Complex numbers	complex numbers

Common Sets

2.2.1 Ordered Pairs

Definition (x, y) is an ordered pair. Two ordered pairs are equal exactly when the first coordinates are equal and the second coordinates are equal.

Key Point Order matters:

$$(a, b) \neq (b, a)$$

in general.

2.2.2 Cartesian Product

Definition

$$X \times Y = \{(x, y) : x \in X, y \in Y\}.$$

Example If $X = \{1, 2\}$ and $Y = \{3, 4, 5\}$, then

$$X \times Y = \{(1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5)\}.$$

2.2.3 Power Set

Definition The power set $\mathcal{P}(A)$ is the set of all subsets of A .

Example For $A = \{1, 2, 3\}$, the power set has 8 subsets, including $\emptyset$, the three one-element subsets, the three two-element subsets, and A itself.

2.2.4 Partitions

Definition A partition of A is a collection of non-empty subsets whose union is A and which are pairwise disjoint.

Example For $A = \{1, 2, 3, 4\}$, both

$$\{\{1\}, \{2, 3\}, \{4\}\}$$

and

$$\{\{1, 2\}, \{3, 4\}\}$$

are valid partitions. A collection whose parts overlap, or which leaves an element uncovered, is not a partition.

2.3 Functions

Definition A function $f : X \rightarrow Y$ assigns each $x \in X$ to exactly one $y \in Y$, written $y = f(x)$.

The set X is the domain, Y is the codomain, and the range is the set of actual output values.

Key Point

$$\text{range}(f) \subseteq \text{codomain}(f).$$

Example A function represented as a set of ordered pairs may be

$$f = \{(a, 2), (b, 4), (c, 2)\}.$$

Each input has exactly one output.

2.3.1 Inverse Image

For $y \in Y$, the inverse image of y is the set of all $x \in X$ such that $f(x) = y$.

Example If the outputs are arranged so that one codomain element has no preimage and another has two preimages, the inverse image is a set rather than one element.

2.3.2 Composition of Functions

Definition For compatible functions f and g ,

$$(g \circ f)(x) = g(f(x)).$$

Always perform f first, then g .

Step-by-Step

1. Start with x .
2. Apply f .
3. Feed the result into g .

Example If $f(n) = n^2$ and $g(n) = n + 1$, then

$$(g \circ f)(n) = n^2 + 1,$$

while

$$(f \circ g)(n) = (n + 1)^2.$$

Therefore composition is not commutative.

Key Point $g \circ f$ is only well-defined when the range of f is contained in the domain of g .

2.3.3 Composition Example

The example defines f on the non-negative reals by $f(x) = 3e^{2x}$ and $g(x) = 2x^2$.

Step-by-Step

$$(g \circ f)(x) = g(f(x)) = 2(3e^{2x})^2 = 18e^{4x}.$$

For $x \geq 0$, the minimum occurs at $x = 0$, giving 18. Thus the range is the set of real values at least 18. Because this is a proper subset of the codomain $\mathbb{R}$, the composition is not surjective and therefore has no inverse function.

2.4 Injections and Surjections

Definition A function is **injective** if no two distinct inputs have the same output.

Step-by-Step Standard proof pattern

1. Assume $f(x_1) = f(x_2)$.
2. Algebraically simplify the equality.
3. If the result forces $x_1 = x_2$, the function is injective.

Definition A function is **surjective** if every element of its codomain is attained. Equivalently, $\text{range} = \text{codomain}$.

1. To prove surjectivity, show that for every y in the codomain, there exists an x in the domain such that $f(x) = y$.
2. To disprove surjectivity, find one element of the codomain that is not an output of the function.

Definition A **bijection** is both injective and surjective.

2.4.1 Identity Function

$$I_X(x) = x.$$

The identity function is a bijection.

2.4.2 Inverse Function

A bijection has an inverse. The inverse reverses the original mapping.

Step-by-Step Finding an inverse

1. Start from $y = f(x)$.
2. Solve for x in terms of y .
3. Rename the resulting expression as $f^{-1}(y)$.

Example

$$y = 3e^{2(x-1)}.$$

Then

$$\ln\left(\frac{y}{3}\right) = 2(x-1),$$

so

$$x = \frac{1}{2} \ln\left(\frac{y}{3}\right) + 1.$$

Hence

$$f^{-1}(y) = \frac{1}{2} \ln\left(\frac{y}{3}\right) + 1.$$

2.4.3 Important Counterexample

It emphasizes that a function such as $f(x) = x^2$ on all of $\mathbb{R}$ is not invertible because it is not injective.

2.4.4 A Floor-Function Example

The example defines

$$f(n) = \left\lfloor \frac{3n+1}{3} \right\rfloor$$

from non-negative integers to non-negative integers.

1. Determine whether f is injective.
2. Determine whether f is surjective.

Because $f(0) = 1$ and the function never reaches 0, it is not surjective.

$$\begin{aligned} \frac{3n+1}{3} &= n + \frac{1}{3} \\ f(n) &= \left\lfloor n + \frac{1}{3} \right\rfloor = n \end{aligned}$$

Therefore distinct inputs have distinct outputs, so the function is injective.

2.4.5 Modulo-10 Function

The given test asks whether

$$f : \mathbb{Z} \rightarrow \mathbb{Z}, \quad f(n) = n \bmod 10$$

is an injection.

Step-by-Step Disproof by counterexample

$$f(1) = 1, \quad f(11) = 1.$$

Since $1 \neq 11$ but the outputs are equal, f is not injective.

2.5 Countable and Uncountable Sets

Definition

1. A finite set has a finite number of elements. Its cardinality is written $|S|$.
2. Two sets have the same cardinality if there is a bijection between them.
3. $|A| \leq |B|$ when there is an injection from A to B .
4. A set is countable when it is finite or can be put in bijection with the natural numbers. A set that is not countable is uncountable.

2.5.1 Countable Examples

The example shows that the even numbers are countable because the mapping $n \mapsto 2n$ provides a one-to-one correspondence with the natural numbers.

The integers are also countable. The example presents an explicit listing argument.

The positive rational numbers are countable by an enumeration in which fractions are arranged systematically; repeated representations can be handled so that the set can be listed.

2.5.2 Union of Countable Sets

Definition Theorem If A and B are countable, then $A \cup B$ is countable.

Step-by-Step Idea for proof Interleave the two countable listings and skip duplicates when necessary.

2.5.3 Uncountability of an Interval

We use Cantor's diagonal argument to show that $(0, 1)$ is uncountable.

Step-by-Step Diagonal idea

1. Assume all elements of $(0, 1)$ can be listed.
2. Construct a new decimal whose n -th digit differs from the n -th digit of the n -th listed number.
3. The constructed decimal differs from every listed number in at least one digit.
4. Therefore the list was not complete, giving a contradiction.

Then we state that $\mathbb{R}$ is uncountable and that the irrational numbers $\mathbb{R} \setminus \mathbb{Q}$ are uncountable.

2.6 Hash Functions

Definition A hash function H maps an input of arbitrary length to a fixed-length output called a hash or digest.

Uses stated Integrity checking, fast lookup, and security applications.

	Cryptographic	Non-cryptographic
Security	High	Low
Speed	Moderate	Very fast
Uses	Password hashing, digital signatures	Hash tables, indexing, checksum
Examples	SHA-256, SHA-3	MurmurHash, DJBX33A

2.6.1 Overflow

This section explains that when the computed value exceeds the 32-bit unsigned limit, the arithmetic wraps around by effectively retaining the least significant 32 bits, which is equivalent to reducing modulo 2^{32} .

2.6.2 Why a Fixed-Length Hash Cannot Be an Injection Here

This section observes that arbitrary-length input strings form a countably infinite collection, while a fixed 32-bit output has only finitely many possible values. Therefore distinct inputs must eventually collide, so the described hash cannot be injective.

3 Unit 3 — Relations

3.1 Definition of Relations

Definition A binary relation R from a set A to a set B is a subset of $A \times B$. A binary relation on A is a subset of A^2 .

We write

$$xRy \iff (x, y) \in R.$$

Example The marriage relation is a subset of the set of men in Hong Kong crossed with the set of women in Hong Kong. Another example is the less-than relation on $\mathbb{R}$, where (x, y) belongs to the relation exactly when $x < y$.

Example Modulo-2 relation on $\mathbb{Z}$:

$$mEn \iff m - n \text{ is even.}$$

Thus $4E6$ is true, while $5E2$ is false. The integers related to 1 are exactly the odd integers.

3.1.1 Relation versus Function

A function is a special kind of relation: for each input there is exactly one output. Every function is a relation, but not every relation is a function.

3.1.2 Inverse Relation

Definition If $R \subseteq A \times B$, then the inverse relation is

$$R^{-1} = \{(y, x) : (x, y) \in R\}.$$

It is obtained by swapping each ordered pair.

Example If R contains $(2, 6)$, then R^{-1} contains $(6, 2)$.

3.1.3 Composition of Relations

If $R \subseteq A \times B$ and $S \subseteq B \times C$, then

$$a(S \circ R)c$$

when there exists some $b \in B$ such that aRb and bSc .

Step-by-Step Apply R first, then S , just as in composition of functions.

3.1.4 k -ary Relations

A k -ary relation is a subset of

$$A_1 \times A_2 \times \cdots \times A_k.$$

Binary means $k = 2$, ternary means $k = 3$, and unary means a subset of one set.

This section examples include prime numbers as a unary relation, twin prime pairs $(3, 5)$, $(5, 7)$, $(11, 13)$ as examples of a binary relation, and Pythagorean triples such as $(3, 4, 5)$ as a ternary relation.

3.1.5 Directed Graph Representation

For a relation on A , draw an arrow $x \rightarrow y$ when xRy .

Example Use even-difference divisibility on $A = \{3, 4, 5, 6, 7, 8\}$ to illustrate a directed graph.

3.2 Properties of Relations

Definition A relation R on A is:

- **Reflexive:** xRx for every $x \in A$.
- **Symmetric:** $xRy \Rightarrow yRx$.
- **Transitive:** xRy and $yRz \Rightarrow xRz$.
- **Antisymmetric:** xRy and $yRx \Rightarrow x = y$.

Step-by-Step How to test a relation Check each property independently. To disprove a property, one counterexample is enough.

Example Less-than $<$ on $\mathbb{R}$ is not reflexive, is not symmetric, and is transitive. Use less-than-or-equal-to as the example for antisymmetry: if $m \leq n$ and $n \leq m$, then $m = n$.

3.2.1 Congruence Modulo 3

The relation aRb when $a - b$ is divisible by 3 is used in this section as an example of an equivalence relation. Reflexivity, symmetry and transitivity are each checked directly from integer arithmetic.

3.2.2 Divisibility as a Relation

The relation $xRy \iff x \mid y$ is reflexive, antisymmetric and transitive, but not symmetric. This connection is later used in Unit 4 when partial order language is introduced.

3.3 Equivalence Relations

Definition An equivalence relation is a relation that is simultaneously reflexive, symmetric and transitive.

3.3.1 Motivation

Use different representations of the same rational number, such as

$$\frac{1}{2} = \frac{2}{4} = \frac{3}{6} = \frac{4}{8},$$

to motivate the idea of grouping objects that are different in form but equivalent in the intended sense.

3.3.2 Partition-Induced Relation

Given a partition P of A , define

$$xRy \iff x \text{ and } y \text{ are in the same part of } P.$$

It follows that this induced relation is an equivalence relation.

3.3.3 Test 1 Partition Example

Worked example

$$A = \{a, b, c, d, e\}, \quad P = \{\{a, b\}, \{c, d\}, \{e\}\}.$$

The relation consists of all ordered pairs lying within the same part:

$$R = \{(a, a), (a, b), (b, a), (b, b), \\ (c, c), (c, d), (d, c), (d, d), (e, e)\}.$$

Because it is induced by a partition, the relation is an equivalence relation.

3.4 Equivalence Classes

Definition For an equivalence relation R on A , the equivalence class of a is

$$[a] = \{x \in A : xRa\}.$$

Because of reflexivity, $a \in [a]$.

Key Point Any two equivalence classes are either equal or disjoint. The full set of equivalence classes forms a partition.

Example For the parity relation on suitable integer sets, there are two classes: even and odd numbers.

Example For congruence modulo 3, the classes are

$$[0] = \{\dots, -6, -3, 0, 3, 6, \dots\},$$

$$[1] = \{\dots, -5, -2, 1, 4, 7, \dots\},$$

$$[2] = \{\dots, -4, -1, 2, 5, 8, \dots\}.$$

Also $[3] = [0]$.

3.4.1 Proof of the Disjoint-or-Equal Property

Suppose two equivalence classes overlap. Let an element belong to both classes. Using symmetry and transitivity, this section shows that every element in one class must belong to the other, hence the classes are equal.

3.4.2 Equivalence Class Example

For

$$f(x, y) = \sqrt{x^2 + y^2}$$

and the relation

$$(x, y)S(p, q) \iff f(x, y) = f(p, q),$$

the class of $(1, 1)$ is

$$[(1, 1)] = \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 2\}.$$

It is a circle centered at the origin with radius $\sqrt{2}$.

3.5 More Examples of Equivalence Relations

3.5.1 Binary Sequences by Parity of Number of 1s

For binary sequences of length n , define xRy when x and y contain either both an odd number of 1s or both an even number of 1s. The example gives $(111, 010)$ as an example of related sequences for $n = 3$.

3.5.2 Logic Circuit Equivalence

For the set of all two-input, one-output logic circuits, define $c_1 R c_2$ when they have the same input/output table. The relation is an equivalence relation. With two inputs and one output, this section notes $2^4 = 16$ equivalence classes.

3.5.3 Calendar Modulo 7

This section models days in a month using congruence modulo 7. For example,

$$[1] = \{1, 8, 15, 22, 29\},$$

and the given calendar associates the seven classes with Thursday through Wednesday. All dates with the same remainder modulo 7 form one equivalence class.

3.5.4 Pure Mathematics Example

On $\mathbb{R}^2$, define

$$(x_1, y_1) R (x_2, y_2) \iff x_1 - x_2 = n \text{ for some } n \in \mathbb{Z}.$$

We can prove this directly: this is an equivalence relation. An equivalence class consists of points whose x -coordinates differ from a reference point by an integer. For example,

$$[(2, 1)] = \{(x, y) : x - 2 \in \mathbb{Z}\}.$$

3.5.5 Relation Example

Worked example

$$(x, y) R (p, q) \iff x - y = p - q.$$

It is reflexive, symmetric and transitive. Each equivalence class satisfies

$$x - y = c$$

for some constant c , so each class is a straight line of slope 1. The class through a point whose difference is -2 is

$$x - y = -2,$$

or equivalently $y = x + 2$.

4 Unit 4 — Divisibility and Euclidean Algorithm

4.1 Number Theory and Cryptography

Number theory studies integers and operations on them. This section emphasizes its importance in modern cryptography, including secure communications, online transactions and e-banking.

4.2 Divisibility

Definition For integers n and d , we say d divides n if there exists $k \in \mathbb{Z}$ such that

$$n = dk.$$

Notation:

$$d \mid n.$$

Then d is a factor or divisor of n , and n is a multiple of d .

Example Determine whether $3 \mid 12$.

Step-by-Step

1. Start from the definition: $3 \mid 12$ means that $12 = 3k$ for some integer k .
2. Compute $12 \div 3 = 4$.
3. Since $4 \in \mathbb{Z}$, the required integer k exists.
4. Therefore, $12 = 3 \times 4$, so $3 \mid 12$.

4.2.1 Divisibility as a Relation

Define $xRy \iff x \mid y$.

Step-by-Step

1. Reflexive: $x \mid x$ because $x = x \times 1$.
2. Antisymmetric: if $x \mid y$ and $y \mid x$, then the only possible positive scaling between them is 1, so $x = y$.
3. Not symmetric: $2 \mid 4$, but $4 \nmid 2$.
4. Transitive: if $x \mid y$ and $y \mid z$, write $y = xk$ and $z = y\ell$. Then $z = x(k\ell)$, so $x \mid z$.

Thus divisibility is an example of a partial order in the material.

4.2.2 Quotient-Remainder Theorem

Definition For an integer n and positive integer d , there are unique integers q and r such that

$$n = dq + r, \quad 0 \leq r < d.$$

Here q is the quotient and r is the remainder.

Step-by-Step Reading a division For positive integers, repeatedly subtract groups of size d until the remainder is smaller than d .

Example Find the quotient and remainder when 29 is divided by 6.

Step-by-Step

$$29 = 6(4) + 5$$

$$q = 4$$

$$r = 5.$$

Check the condition:

$$0 \leq 5 < 6.$$

Therefore,

$$29 \operatorname{div} 6 = 4, \quad 29 \operatorname{mod} 6 = 5.$$

4.3 Primes and Co-primes

Definition A prime number is an integer $p > 1$ whose only positive divisors are 1 and p .

Definition A composite number is an integer $n > 1$ that is not prime.

Key Point 1 is not prime.

Example Classify 1, 2 and 4.

Step-by-Step

1. 1 is not prime because the definition requires $p > 1$.
2. 2 has only positive divisors 1 and 2, so 2 is prime.
3. 4 has positive divisors 1, 2, 4. Since $2 \mid 4$ in addition to 1 and 4, 4 is composite.

4.3.1 Infinitely Many Primes

Theorem There are infinitely many primes.

Step-by-Step Euclid's proof

1. Assume there are only finitely many primes $p_1, p_2, \dots, p_k$.

2. Form

$$Q = p_1 p_2 \cdots p_k + 1.$$

3. Let p be a prime divisor of Q .

4. If one of the listed primes, say p_j , divided Q , then p_j would also divide

$$Q - p_1 p_2 \cdots p_k = 1,$$

which is impossible.

5. Therefore p is a prime not in the original list.

6. This contradicts the assumption that the list contained all primes.

Hence there must be infinitely many primes.

4.3.2 Greatest Common Divisor

Definition $\gcd(a, b)$ is the largest positive integer dividing both a and b .

Example Find $\gcd(24, 16)$.

Step-by-Step

1. List the positive divisors of 24:

$$1, 2, 3, 4, 6, 8, 12, 24.$$

2. List the positive divisors of 16:

$$1, 2, 4, 8, 16.$$

3. Find the common divisors:

$$1, 2, 4, 8.$$

4. The largest common divisor is 8.

Therefore,

$$\gcd(24, 16) = 8.$$

Definition Two integers are co-prime if their greatest common divisor is 1.

Example Determine whether 14 and 9 are co-prime.

Step-by-Step

$$\gcd(14, 9) = 1.$$

Therefore 14 and 9 are co-prime.

4.3.3 Euler's Totient Function

Definition $\varphi(n)$ counts the positive integers from 1 to n that are co-prime with n .

Small values include

$$\varphi(1) = 1, \quad \varphi(2) = 1, \quad \varphi(3) = 2, \quad \varphi(4) = 2, \quad \varphi(5) = 4, \quad \varphi(6) = 2.$$

For a prime p ,

$$\varphi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1).$$

If m and n are co-prime,

$$\varphi(mn) = \varphi(m)\varphi(n).$$

Example Find $\varphi(12)$ directly.

Step-by-Step

1. The positive integers from 1 to 12 are

$$1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12.$$

2. Keep the values co-prime with 12:

$$1, 5, 7, 11.$$

3. There are four such values.

Therefore,

$$\varphi(12) = 4.$$

Worked Example Totient equation Suppose

$$\varphi(p^2) = 272.$$

Find the prime p .

Step-by-Step

1. Since p is prime,

$$\varphi(p^2) = p(p - 1).$$

2. Substitute the given value:

$$p(p - 1) = 272.$$

3. Expand:

$$p^2 - p - 272 = 0.$$

4. Factor or solve the quadratic equation:

$$(p - 17)(p + 16) = 0.$$

5. Therefore $p = 17$ or $p = -16$.

6. A prime number is positive and greater than 1, so reject -16 .

Hence,

$$p = 17.$$

Now suppose

$$\varphi(pq^2) = 672.$$

With $p = 17$ and co-prime primes p, q ,

$$\varphi(pq^2) = (p - 1)q(q - 1).$$

Thus

$$16q(q - 1) = 672,$$

so

$$q(q - 1) = 42,$$

$$q^2 - q - 42 = 0,$$

$$(q - 7)(q + 6) = 0.$$

Reject $q = -6$ and obtain

$$q = 7.$$

4.4 Euclidean Algorithm

4.4.1 Three Ways to Compute a GCD

Possible methods are:

1. Prime factorization.
2. Search through possible divisors.
3. Euclidean algorithm.

The Euclidean algorithm is the efficient method used throughout the material.

4.4.2 Euclidean Algorithm

Theorem If

$$a = bq + r,$$

then

$$\gcd(a, b) = \gcd(b, r).$$

Step-by-Step Procedure

1. Start with $a \geq b \geq 0$.
2. Divide a by b and record the remainder r .
3. Replace (a, b) by (b, r) .
4. Continue until the remainder becomes 0.
5. The previous non-zero remainder is the gcd.

Example Compute $\gcd(416, 39)$.

Step-by-Step Step-by-step calculation

$$416 = 39(10) + 26,$$

$$39 = 26(1) + 13,$$

$$26 = 13(2) + 0.$$

Step	Dividend	Divisor	Remainder
1	416	39	26
2	39	26	13
3	26	13	0

The last non-zero remainder is 13, so

$$\boxed{\gcd(416, 39) = 13}.$$

Why it works If $a = bq + r$, every common divisor of a and b also divides $r = a - bq$. Conversely, every common divisor of b and r also divides $a = bq + r$. Therefore the two pairs have exactly the same common divisors and hence the same gcd.

4.5 Extended Euclidean Algorithm

Definition A Bézout identity expresses the gcd as

$$\gcd(a, b) = ax + by$$

for some integers x, y . The coefficients are called Bézout coefficients and are not unique in general.

Certificate Lemma If d divides both a and b and

$$d = ax + by$$

for integers x, y , then $d = \gcd(a, b)$.

Step-by-Step Extended Euclidean Algorithm Start with the two original numbers. For each new remainder, keep the coefficients of the original a and b .

Worked Example $\gcd(240, 46)$

Step-by-Step 1. Ordinary Euclidean divisions

$$240 = 46(5) + 10,$$

$$46 = 10(4) + 6,$$

$$10 = 6(1) + 4,$$

$$6 = 4(1) + 2,$$

$$4 = 2(2) + 0.$$

Step-by-Step 2. Full coefficient table

Row	x	y	r	Relation
A	1	0	240	$240 = 240(1) + 46(0)$
B	0	1	46	$46 = 240(0) + 46(1)$
C	1	-5	10	$10 = 240(1) + 46(-5)$
D	-4	21	6	$6 = 240(-4) + 46(21)$
E	5	-26	4	$4 = 240(5) + 46(-26)$
F	-9	47	2	$2 = 240(-9) + 46(47)$

Step-by-Step 3. Read the final non-zero row The final non-zero remainder is 2, so

$$\gcd(240, 46) = 2.$$

The same row gives

$$2 = 240(-9) + 46(47).$$

Therefore,

$$x = -9, \quad y = 47.$$

Worked Example modular inverse of 18 modulo 97

Step-by-Step 1. Apply Euclid

$$97 = 18(5) + 7,$$

$$18 = 7(2) + 4,$$

$$7 = 4(1) + 3,$$

$$4 = 3(1) + 1,$$

$$3 = 1(3) + 0.$$

Step-by-Step 2. Full coefficient table

Row	x	y	r	Relation
A	1	0	97	$97 = 97(1) + 18(0)$
B	0	1	18	$18 = 97(0) + 18(1)$
C	1	-5	7	$7 = 97(1) + 18(-5)$
D	-2	11	4	$4 = 97(-2) + 18(11)$
E	3	-16	3	$3 = 97(3) + 18(-16)$
F	-5	27	1	$1 = 97(-5) + 18(27)$

Step-by-Step 3. Convert the Bézout identity into an inverse From the last row,

$$1 = 97(-5) + 18(27).$$

Reduce modulo 97:

$$18(27) \equiv 1 \pmod{97}.$$

Hence,

$$\boxed{18^{-1} \equiv 27 \pmod{97}}.$$

4.6 Fundamental Theorem of Arithmetic

Theorem Every integer $n > 1$ can be expressed as a product of prime powers, and the representation is unique apart from the order of the factors:

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}.$$

This is also called the Unique Factorization Theorem.

Example Factor 1001 into primes.

Step-by-Step

1. Start with

$$1001 = 7 \times 143.$$

2. Factor 143:

$$143 = 11 \times 13.$$

3. Therefore,

$$1001 = 7 \times 11 \times 13.$$

4. A different route,

$$1001 = 11 \times 91 = 11 \times 7 \times 13,$$

gives exactly the same prime factors.

Why 1 is not prime If 1 were prime, prime factorizations would not be unique because arbitrary extra factors of 1 could be inserted.

4.6.1 Euclid's Lemma

Theorem If p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.

The result follows from Bézout's identity and extends to products of several factors.

5 Unit 5 — Modular Arithmetic and Chinese Remainder Theorem

5.1 What is Modular Arithmetic?

Definition

$$a \equiv b \pmod{n}$$

means that a and b have the same remainder when divided by n .

5.1.1 Field Idea

Informally, a field is a set in which addition, subtraction, multiplication and division are properly defined, except division by zero. The formal description included in this section says that F is a commutative group under addition, $F \setminus \{0\}$ is a commutative group under multiplication, and distributivity holds.

$\mathbb{R}$ and $\mathbb{C}$ are examples, while $\mathbb{N}$ and $\mathbb{Z}$ are non-examples. Rational numbers are included as a prompt in this section.

Structure	Operations emphasized	Example(s) in this section
Group	Addition/subtraction structure	24-hour clock example
Ring	Addition, subtraction, multiplication	$\mathbb{Z}$; modulo n for composite n
Field	Addition, subtraction, multiplication, division except by 0	$\mathbb{Q}, \mathbb{R}, \mathbb{C}$; modulo p for prime p
Vector space	Vector addition/subtraction and scalar multiplication	Real, complex and binary vector spaces

5.1.2 Arithmetic under Congruence

If

$$a \equiv c \pmod{n}, \quad b \equiv d \pmod{n},$$

then

$$a + b \equiv c + d \pmod{n},$$

$$a - b \equiv c - d \pmod{n},$$

$$ab \equiv cd \pmod{n}.$$

5.2 Diophantine Equations

Definition A Diophantine equation is an equation for which integer solutions are required. The unit focuses on linear two-variable equations

$$ax + by = c.$$

Example The example gives a cash-payment interpretation using 20-dollar and 50-dollar notes for a 230-dollar payment.

Theorem The equation

$$ax + by = c$$

has an integer solution if and only if

$$\gcd(a, b) \mid c.$$

Step-by-Step How to solve when a solution exists

1. Compute $d = \gcd(a, b)$ using extended Euclid.
2. Find x_0, y_0 such that $ax_0 + by_0 = d$.
3. Multiply the Bézout identity by c/d .
4. The resulting pair is one particular integer solution.

Example For

$$391x + 299y = -69,$$

the example gives

$$391(-3) + 299(4) = 23.$$

Multiplying by -3 yields

$$391(9) + 299(-12) = -69.$$

Thus one solution is

$$(x, y) = (9, -12).$$

This section also gives another solution, $(x, y) = (-4, 5)$.

5.3 Modular Division

5.3.1 Why division is different

In modular arithmetic, dividing by a means multiplying by the multiplicative inverse of a , when the inverse exists. This section illustrates that, modulo 7, every non-zero row of the multiplication table contains all residues, so equations $ax \equiv b \pmod{7}$ are solvable for non-zero a .

Definition a^{-1} modulo n is a number satisfying

$$aa^{-1} \equiv 1 \pmod{n}.$$

Theorem The inverse $a^{-1} \pmod{n}$ exists if and only if

$$\gcd(a, n) = 1.$$

Step-by-Step Finding an inverse

1. Use extended Euclid to obtain

$$ax + ny = 1.$$

2. Reduce the coefficient x modulo n .
3. The resulting residue is $a^{-1} \pmod{n}$.

Example

$$2^{-1} \equiv 4 \pmod{7}$$

because $2 \times 4 = 8 \equiv 1 \pmod{7}$. Therefore

$$2x \equiv 3 \pmod{7}$$

can be solved by multiplying both sides by 4:

$$x \equiv 12 \equiv 5 \pmod{7}.$$

Example In modulo 6, 2^{-1} does not exist because $\gcd(2, 6) = 2 \neq 1$. Hence the congruence $2x \equiv 5 \pmod{6}$ has no solution.

5.3.2 Division Modulo a Prime

When the modulus is a prime p , every non-zero element has a multiplicative inverse. This section connects this with the fact that arithmetic modulo a prime forms a field.

5.4 Modular Exponentiation

Direct computation of a huge power can be impractical. This section presents square-and-multiply.

Step-by-Step Square-and-Multiply

1. Write the exponent in binary or as a sum of powers of 2.
2. Compute

$$m, m^2, m^4, m^8, \dots$$

reducing modulo n after every squaring.

3. Multiply only the powers corresponding to the 1-bits in the exponent.
4. Reduce modulo n again.

Example Compute

$$17^{29} \pmod{35}.$$

This section values are

$$17^2 \equiv 9, \quad 17^4 \equiv 11, \quad 17^8 \equiv 16, \quad 17^{16} \equiv 11 \pmod{35}.$$

Since

$$29 = 16 + 8 + 4 + 1,$$

combine the corresponding powers modulo 35 to obtain

$$17^{29} \equiv 12 \pmod{35}.$$

5.4.1 Fermat's Little Theorem

Theorem If p is prime and $p \nmid a$, then

$$a^{p-1} \equiv 1 \pmod{p}.$$

Example

$$2^{35} \pmod{7}.$$

Because $2^6 \equiv 1 \pmod{7}$, the exponent can be reduced modulo 6, giving

$$2^{35} \equiv 2^5 \equiv 4 \pmod{7}.$$

Example This section also reduces $40^{110} \pmod{37}$ by first noting $40 \equiv 3 \pmod{37}$ and then using $3^{36} \equiv 1 \pmod{37}$, giving

$$40^{110} \equiv 3^{110} \equiv 3^2 \equiv 9 \pmod{37}.$$

Example Solve

$$x^{103} \equiv 4 \pmod{11}.$$

Since $x^{10} \equiv 1 \pmod{11}$ for non-zero x , reduce the exponent to 3:

$$x^{103} \equiv x^3 \equiv 4 \pmod{11}.$$

This section then tests residues to obtain $x \equiv 5 \pmod{11}$.

5.4.2 Euler's Theorem

Theorem If $\gcd(a, n) = 1$, then

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Unlike Fermat's theorem, the modulus need not be prime.

5.5 Chinese Remainder Theorem

5.5.1 Motivating Examples

This section starts with congruences such as

$$x \equiv 1 \pmod{2}, \quad x \equiv 3 \pmod{5},$$

which combine to

$$x \equiv 3 \pmod{10}.$$

It also contrasts a co-prime case with congruences such as modulo 2 and modulo 4.

Theorem Let m and n be co-prime. Then a pair

$$x \equiv a \pmod{m}, \quad x \equiv b \pmod{n}$$

has a unique solution modulo mn .

Step-by-Step CRT construction Suppose extended Euclid gives

$$m\alpha + n\beta = 1.$$

Then the example gives the solution in the form

$$c \equiv a n\beta + b m\alpha \pmod{mn}.$$

Example Solve

$$x \equiv 3 \pmod{19}, \quad x \equiv 8 \pmod{11}.$$

The given Euclidean coefficients are

$$19(-4) + 11(7) = 1.$$

Hence

$$x \equiv 3(11)(7) + 8(19)(-4) \pmod{209},$$

so this section result is

$$x \equiv 41 \pmod{209}.$$

5.5.2 Sunzi's 3-5-7 Problem

It follows that

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}, \quad x \equiv 2 \pmod{7}.$$

The combined solution is represented modulo 105.

Step-by-Step First combine the first two equations using

$$3(2) + 5(-1) = 1.$$

Then combine the result with the modulo-7 equation using the given Bézout relation

$$15(1) + 7(-2) = 1.$$

A compact expression is

$$x \equiv 70a_1 + 21a_2 + 15a_3 \pmod{105},$$

where the three residues are the given values.

5.5.3 Why CRT Matters Later

The key point is CRT as useful in RSA correctness and in combining modular conditions efficiently.

5.6 SageMath

SageMath is free mathematical software built on Python. SageMathCell provides a browser-based way to use it.

Command	Purpose stated in this section
<code>factor(x)</code>	Factor an integer
<code>nth_prime(n)</code>	Find the n -th prime
<code>gcd(a,b)</code>	Compute a gcd
<code>xgcd(a,b)</code>	Extended gcd
<code>euler_phi(x)</code>	Compute Euler's totient
<code>mod(a,n)</code>	Create/use a modular arithmetic object
<code>inverse_mod(x,y)</code>	Compute a modular inverse
<code>power_mod(x,y,z)</code>	Compute modular powers

6 Unit 6 — Public Key Cryptography

6.1 Cybersecurity Ethics

Definition Ethics comes from the Greek word *ethos*, originally meaning custom or habit. It is the branch of philosophy concerned with human conduct and with questions of right/wrong and just/unjust behavior.

6.1.1 Codes of Ethics

In professional life, ethics is often stated in formal codes or standards. They describe values and principles to which members of a profession aspire and by which their actions may be judged.

6.1.2 (ISC)² Code of Ethics

Consider the four canons used by (ISC)² for CISSP certification holders:

1. Protect society, the common good, necessary public trust and confidence, and the infrastructure.
2. Act honorably, honestly, justly, responsibly, and legally.
3. Provide diligent and competent service to principals.
4. Advance and protect the profession.

The given slide arranges these from the highest standard (society) to the lowest (the profession).

6.1.3 Cybersecurity Example

Consider a professional securing a hospital network and critical data. Patient privacy, health and even survival can depend on the success or failure of the security work.

6.1.4 Types of Hackers

Black hat	Criminals who break into computer networks with malicious intent.
White hat	Ethical hackers / penetration testers who have permission to identify security flaws.
Grey hat	Not necessarily malicious, but accesses systems without permission and may violate laws or ethical standards; may report vulnerabilities and sometimes request a fee.

6.2 Public Key Cryptography

6.2.1 Two Problems Motivating Public-Key Cryptography

1. How can Alice and Bob share a secret key for symmetric cryptography?
2. How can Bob be prevented from denying that he sent a certain message?

6.2.2 Two Main Motivations

1. **Key distribution at scale:** anyone can encrypt to a public key without a prior shared secret.
2. **Authentication and non-repudiation:** digital signatures let a party prove data origin and integrity.

6.2.3 Public-Key Requirements

Consider a public-key mechanism in which:

- encryption is easy,
- decryption by an unauthorized party is hard,
- the legitimate receiver can decrypt using a private key.

6.2.4 Trapdoor One-Way Functions

Definition Consider a function that is easy to compute in the forward direction but computationally hard to invert without special trapdoor information. With the trapdoor, inversion becomes easy.

6.2.5 Three Families Mentioned

- Integer factorization: RSA.
- Discrete logarithm: Diffie-Hellman and El Gamal.
- Elliptic-curve schemes based on the elliptic-curve discrete logarithm problem.

6.2.6 Why Public-Key Systems Are Usually Used to Set Up a Session Key

This section emphasizes that public-key operations are more expensive, so public-key systems are commonly used initially to establish a common private/session key, after which symmetric cryptography handles the main data stream.

6.2.7 HTTPS and SSL/TLS Workflow

A simplified workflow is:

1. The client requests an HTTPS connection.
2. The server supplies a certificate containing its public key.
3. The client creates a session key k_S .
4. The client encrypts k_S using the server public key.
5. The server decrypts it using its private key.
6. Both sides now share k_S for symmetric encryption.

The material describes HTTPS as HTTP plus SSL/TLS. The given history table lists SSL 1.0 as never publicly released because of a serious flaw; SSL 2.0 (1995), SSL 3.0 (1996), TLS 1.0 (1999), TLS 1.1 (2006), TLS 1.2 (2008), and TLS 1.3 as “Upcoming” in the material.

6.2.8 Digital Signatures

A simplified signature scheme in this section has Bob use his private key to create a signature associated with message m . Alice uses Bob’s public key to verify it.

Step-by-Step Verification logic

1. Bob creates the signature using his private key.
2. Bob sends the message together with the signature.
3. Alice uses Bob’s public key to verify.
4. A valid result supports origin authentication and non-repudiation.

6.2.9 Hash-Then-Sign

Signing a long message directly can be computationally expensive. This section’s solution is to hash the message and sign the fixed-length hash:

$$\text{signature} = f(H(m), k_A^-).$$

The receiver computes $H(m)$ independently and compares it with the recovered signed hash.

6.3 RSA Cryptosystems

6.3.1 Background

RSA was developed by Rivest, Shamir and Adleman and dates its invention to 1977. Its security discussion is based on the difficulty of factoring large integers. The material also shows an RSA challenge involving a 617-digit, 2048-bit integer as an illustration of the scale of the numbers involved.

6.3.2 Preparing the Message

This section represents a bit string by an integer. For example,

$$10010001_2 = 145.$$

6.3.3 RSA Key Generation

Step-by-Step

1. Choose distinct large random primes p and q .

2. Compute

$$N = pq.$$

3. Compute

$$\varphi(N) = (p - 1)(q - 1).$$

4. Choose e such that

$$1 < e < \varphi(N), \quad \gcd(e, \varphi(N)) = 1.$$

5. Find d such that

$$ed \equiv 1 \pmod{\varphi(N)}.$$

6. Publish

$$K_B^+ = (N, e),$$

and keep

$$K_B^- = (N, d).$$

6.3.4 RSA Encryption and Decryption

$$c \equiv m^e \pmod{N},$$

then

$$m' \equiv c^d \pmod{N}.$$

This section assumes $m < N$, with longer messages broken into blocks when required.

Example Toy RSA example from this section Take

$$p = 5, \quad q = 7,$$

so

$$N = 35, \quad \varphi(N) = 24.$$

Choose

$$e = 5,$$

and the given private exponent is

$$d = 5$$

because $5 \times 5 = 25 \equiv 1 \pmod{24}$.

For message $m = 12$:

$$c \equiv 12^5 \equiv 17 \pmod{35}.$$

Decrypting gives

$$17^5 \equiv 12 \pmod{35}.$$

It is important to the use of fast modular exponentiation for real-sized computations.

6.3.5 Why RSA Works

The goal is

$$(m^e)^d = m^{ed} \equiv m \pmod{N}.$$

We can prove this directly: this separately modulo p and modulo q , then uses the Chinese remainder theorem to conclude the congruence modulo $N = pq$.

6.3.6 Why Factoring Matters for RSA Security

Given N and $\varphi(N)$, one can recover the factors because

$$\varphi(N) = (p - 1)(q - 1) = pq - p - q + 1.$$

Since $pq = N$, knowing N and $\varphi(N)$ gives enough information to solve for p and q . The security idea in this section is therefore tied to the difficulty of factoring N .

6.4 Diffie-Hellman Key Exchange

6.4.1 Discrete Logarithm

For ordinary exponentiation $y = a^x$, x is the discrete logarithm in the modular version

$$y \equiv a^x \pmod{p}.$$

Security relies on the computational difficulty of solving for x .

6.4.2 $GF(p)$, Order and Primitive Elements

Use

$$GF(p) = \{0, 1, \dots, p-1\}$$

with modular arithmetic.

Definition The order $\text{ord}_p(\alpha)$ is the smallest positive k such that

$$\alpha^k \equiv 1 \pmod{p}.$$

The order divides $p-1$.

Definition A primitive element has order $p-1$ and generates all non-zero elements of $GF(p)$.

Example In $GF(7)$,

$$2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 8 \equiv 1 \pmod{7},$$

so

$$\text{ord}_7(2) = 3.$$

6.4.3 Diffie-Hellman Protocol

Step-by-Step

1. Publicly agree on a prime p and primitive element α .
2. Alice chooses random a and sends

$$y_A = \alpha^a \pmod{p}.$$

3. Bob chooses random b and sends

$$y_B = \alpha^b \pmod{p}.$$

4. Alice computes

$$y_B^a \equiv \alpha^{ab} \pmod{p}.$$

5. Bob computes

$$y_A^b \equiv \alpha^{ab} \pmod{p}.$$

Thus both obtain the common key

$$k = \alpha^{ab} \pmod{p}.$$

6.4.4 Why a Passive Eve Has Difficulty

Eve can see p , α , y_A and y_B , but this section argues that recovering a or b requires solving a discrete logarithm. Choosing a primitive element gives the largest possible non-zero secret space.

6.4.5 Man-in-the-Middle Attack

This section also shows that Diffie-Hellman by itself does not authenticate the parties. An active attacker can establish one key with Alice and another with Bob, decrypt traffic on one side and re-encrypt it on the other. This is the given MITM attack.

Worked Example The given solution computes by square-and-multiply for

$$5^{64} \pmod{157},$$

obtaining

$$y_A = 147.$$

Using Bob's exponent 94, the shared key is calculated as

$$147^{94} \pmod{157} = 89.$$

The important exam procedure is repeated squaring plus selecting the powers corresponding to the 1-bits of the exponent.

6.5 El Gamal Cryptosystems

6.5.1 Key Generation

Consider El Gamal as based on Diffie-Hellman and attributes it to Taher Elgamal.

Public parameters:

$$p = \text{prime}, \quad \alpha = \text{primitive element of } GF(p).$$

Alice chooses private value d and computes

$$e = \alpha^d \pmod{p}.$$

Then

$$K_A^- = (p, \alpha, d), \quad K_A^+ = (p, \alpha, e).$$

6.5.2 Encryption

Step-by-Step

1. Represent the message as an integer $0 \leq m < p$.

2. Choose a fresh random k for each block.

3. Compute the one-time key

$$K = e^k \pmod{p}.$$

4. Compute

$$c_1 = \alpha^k \pmod{p}, \quad c_2 = Km \pmod{p}.$$

5. The ciphertext is (c_1, c_2) .

6.5.3 Decryption

Step-by-Step

1. Compute

$$K = c_1^d \pmod{p}.$$

2. Compute

$$m = c_2 K^{-1} \pmod{p}.$$

Example Use $GF(19)$, $\alpha = 10$, Alice's private value $d = 5$, giving $e = 3$. For message $m = 17$ and random $k = 6$:

$$K = 3^6 \equiv 7 \pmod{19},$$

$$c_1 = 10^6 \equiv 11 \pmod{19},$$

$$c_2 = 7 \times 17 \equiv 5 \pmod{19}.$$

During decryption,

$$11^5 \equiv 7 \pmod{19},$$

and the given inverse is

$$7^{-1} \equiv 11 \pmod{19}.$$

Hence

$$5 \times 11 \equiv 17 \pmod{19}.$$

Why decryption recovers the same key Because

$$e = \alpha^d,$$

we have

$$e^k = \alpha^{dk} = (\alpha^k)^d = c_1^d.$$

So the receiver reconstructs the same K used by the sender.

Security statement in this section Eve sees $e = \alpha^d$ and $c_1 = \alpha^k$; this section explains that recovering the hidden exponents involves the discrete logarithm problem.

7 Unit 7 — Digital Certificate

7.1 Motivation: Identity Verification Problem

Begin with the pizza-store prank. Trudy creates a message pretending to be Bob, signs it with her own private key, and then sends her public key while claiming that it is Bob's public key. The store verifies the signature and delivers the pizza to Bob.

Key Point The underlying problem is not simply whether a signature verifies. The receiver must also know whose public key is being used.

7.2 Public-Key Authority

Step-by-Step

1. Bob registers his public key k_B^+ with an authority.
2. The authority's public key k_{AA}^+ is known to everyone.
3. The Pizza Store requests Bob's public key.
4. The authority returns a message protected using the authority's private key k_{AA}^- .
5. The store uses k_{AA}^+ and obtains Bob's public key.

Major drawback Real-time communication with the authority is required whenever the public key is requested.

7.3 Digital Certificates

Definition A digital certificate is an electronic credential issued by an authority to prove ownership/binding of a public key.

The authority is called a **Certificate Authority (CA)** or Certification Authority. The CA's public key is known to users, for example through storage in browsers or caches.

7.4 Issuing a Digital Certificate

Step-by-Step

1. Bob registers his public key with the CA.
2. Bob provides proof of identity.

3. The CA creates a certificate binding Bob's identifying information to Bob's public key.
4. The CA digitally signs the certificate.

The important statement from this section is that the CA is effectively saying, "this is Bob's public key."

7.5 Using a Digital Certificate

Step-by-Step

1. The Pizza Store obtains Bob's certificate.
2. It applies the CA's public key to the CA signature.
3. It obtains Bob's public key from the certificate.
4. No real-time CA communication is needed.

7.6 X.509

Definition X.509 is an ITU standard specifying the structure of public-key certificates. It follows that X.509 certificates are used in protocols such as TLS/SSL for HTTPS, email encryption and VPN authentication.

7.6.1 X.509 Certificate Structure

The given material lists fields such as:

- Version, typically version 3 in the shown example.
- Serial number.
- Signature algorithm identifier.
- Issuer: the CA.
- Validity period.
- Subject: domain/organization identity information.
- Subject Public Key Info.
- Extensions, including examples such as key usage and subject alternative names.

7.7 Self-Signed Certificate

Definition A self-signed certificate is signed by the same entity that it certifies. This section presents this as useful for internal testing, development and private networks.

7.8 OpenSSL Practical Activity

The given TBL material gives the following commands.

```
openssl genrsa -out weakkey.pem 512
openssl req -new -key weakkey.pem -out weak.csr
openssl x509 -req -days 365 -in weak.csr -signkey weakkey.pem -out weakcert.
pem
```

To inspect the certificate:

```
openssl x509 -in weakcert.pem -text -noout
```

To inspect the private key:

```
openssl pkey -in weakkey.pem -text -noout
```

The example output shown in the given material contains fields such as version 3, SHA-256 with RSA, subject/issuer, RSA public-key algorithm, a 512-bit public key, hexadecimal modulus, exponent 65537, X.509v3 Subject Key Identifier, and a signature value. The private-key display contains additional values such as n, e, d, p, q and related exponents/coefficient information.

7.9 TBL Preparation

The given activity asks students to review how digital certificates work and why they are needed, then use an online cryptography tool to create and inspect a self-signed certificate and private key.

8 Unit 8 — Linear Algebra for Cryptography

8.1 Opening Idea: Four-Digit Combination Lock

Begin with with a lock whose combination can be opened by any two children together, while one child alone gets no information and must in effect face the full search space of 10,000 four-digit combinations. This motivates secret sharing.

8.2 Linear Mappings

8.2.1 Vectors

This section presents a vector as the basic building block of linear algebra. A vector can be pictured as an arrow with length and direction, or as an ordered list of n numbers.

Definition Mathematically, a vector is an element of a vector space. The two basic operations emphasized in this section are vector addition and scalar multiplication.

Definition

$$\mathbb{R}^n = \{(x_1, \dots, x_n) : x_i \in \mathbb{R}\}.$$

Note that the underlying field need not always be the real numbers; it can also be the complex numbers or a finite field such as arithmetic modulo a prime.

8.2.2 Vector-Valued Mappings

Consider

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad y = f(x),$$

where both input and output may be vectors. When $m = 1$, the function is scalar-valued.

8.2.3 Linearity

Definition A function f is linear if it satisfies:

1. Additivity:

$$f(x + y) = f(x) + f(y).$$

2. Scaling:

$$f(cx) = cf(x).$$

These combine into the superposition condition

$$f(\alpha x + \beta y) = \alpha f(x) + \beta f(y).$$

Example Exchange-rate illustration The example gives 100 yen = HK\$7.00, with one plate of soba costing 800 yen and sushi 200 yen. For two plates of soba and four plates of sushi,

$$2(800) + 4(200) = 2400 \text{ yen} = 24(7) = \text{HK\$168}.$$

The linear scaling of the total cost is the intended illustration.

8.2.4 Zero-in, Zero-out

A linear map must satisfy

$$f(0) = 0.$$

Use this as an important quick test and asks whether zero-in-zero-out is by itself sufficient for linearity.

8.2.5 Examples and Non-Examples

Consider functions such as averages, coordinate differences and electrical relations such as Ohm's law as linear examples, while functions such as minimum and the Euclidean norm are used to illustrate non-linearity.

8.2.6 Inner Product

For vectors u and v , the inner product is written

$$u^T v = \sum_i u_i v_i.$$

Note that symmetry,

$$a^T b = b^T a,$$

and positive definiteness,

$$a^T a \geq 0,$$

with equality only for the zero vector.

8.2.7 A Linear Function from an Inner Product

Consider

$$f(v) = u^T v$$

and verifies superposition directly.

The Euclidean norm is used as a non-linear example because the corresponding superposition relationship fails.

8.3 Matrix Representation of Linearity

8.3.1 Matrix-Vector Multiplication

Let A be an $m \times n$ matrix and x an n -vector. Then Ax is an m -vector.

Example The given matrix is

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 3 & 2 & 1 \end{pmatrix}, \quad x = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix}.$$

Row-wise computation gives

$$Ax = \begin{pmatrix} 1(2) + 2(-1) + 3(1) \\ 3(2) + 2(-1) + 1(1) \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}.$$

This section also gives the column interpretation: Ax is a linear combination of the columns of A .

8.3.2 Every Matrix Defines a Linear Map

For

$$f(x) = Ax,$$

linearity follows from matrix multiplication:

$$A(\alpha x + \beta y) = \alpha Ax + \beta Ay.$$

8.3.3 Geometric Transformations

The example gives the following matrix examples:

- Stretch: cI .
- Rotation by 90 degrees:

$$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

- Reflection about the 45-degree line:

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

- Projection onto the x -axis:

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

8.3.4 Converse Theorem

It follows that every linear map from $\mathbb{R}^n$ to $\mathbb{R}^m$ can be represented as multiplication by an appropriate matrix. The proof idea uses standard basis vectors and superposition.

8.3.5 Affine Functions

An affine function has the form

$$g(x) = Ax + c.$$

This section emphasizes that, strictly speaking, a non-zero constant shift means the map is not linear because zero-in-zero-out fails, although the term “linear” is sometimes used informally.

8.3.6 Composition of Linear Maps

If

$$f(x) = Ax, \quad g(y) = By,$$

then

$$(g \circ f)(x) = BAx,$$

so the composition is again linear.

8.4 Determinants and Matrix Inverses

8.4.1 Determinant of a 2×2 Matrix

Definition For

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix},$$

$$\det A = a_{11}a_{22} - a_{12}a_{21}.$$

8.4.2 Minor and Cofactor

Definition The minor M_{ij} is the determinant obtained by deleting row i and column j .

Definition The cofactor is

$$C_{ij} = (-1)^{i+j} M_{ij}.$$

Example For the given matrix

$$A = \begin{pmatrix} 3 & -1 & 6 \\ 9 & -5 & 2 \\ 0 & 4 & 7 \end{pmatrix},$$

the example gives $M_{23} = 12$, hence

$$C_{23} = -12.$$

8.4.3 Cofactor Expansion

For a square matrix, expand along any row:

$$\det A = a_{i1}C_{i1} + a_{i2}C_{i2} + \cdots + a_{in}C_{in}.$$

For the given 3×3 matrix, the example gives determinant 150 and the first-row cofactors

$$C_{11} = -43, \quad C_{12} = -63, \quad C_{13} = 36.$$

8.4.4 Cramer's Rule

For n linear equations in n unknowns, if

$$\det A \neq 0,$$

then Cramer's rule gives each variable by replacing one column of A and dividing the resulting determinant by $\det A$.

8.4.5 Matrix Inverse by Cofactors

Step-by-Step

1. Compute all minors.
2. Convert minors to cofactors.
3. Transpose the cofactor matrix to obtain the adjugate/adjoint matrix.
4. Divide by $\det A$.

Thus

$$A^{-1} = \frac{1}{\det A} \text{adj}(A).$$

For the given 3×3 matrix, the example gives

$$\text{adj}(A) = \begin{pmatrix} -43 & 31 & 28 \\ -63 & 21 & 48 \\ 36 & -12 & -6 \end{pmatrix}$$

and $\det A = 150$.

Note that larger matrices are better handled by Gaussian elimination or software such as MATLAB or SageMath.

8.5 Symmetric Ciphers

8.5.1 Affine Cipher

Definition The given affine cipher is

$$c \equiv \alpha m + \beta \pmod{b}.$$

For decryption,

$$m \equiv \alpha^{-1}(c - \beta) \pmod{b},$$

provided $\gcd(\alpha, b) = 1$. When $\beta = 0$, the cipher is linear.

8.5.2 Hill Cipher

Consider the Hill cipher as a polygraphic substitution cipher due to Lester Hill. Letters are grouped into blocks and represented as vectors. An invertible secret matrix E is used:

$$c = Em$$

under the appropriate modular arithmetic, with decryption using

$$m = E^{-1}c.$$

8.5.3 Invertibility Modulo 26

It follows that the determinant must have a modular inverse. In particular, the determinant should be co-prime with the modulus.

Example The given key is

$$E = \begin{pmatrix} 6 & 24 & 1 \\ 13 & 16 & 10 \\ 20 & 17 & 15 \end{pmatrix}.$$

The example gives

$$\det E = 441 \equiv 25 \pmod{26},$$

and since 25 is co-prime to 26, the matrix is invertible modulo 26.

For plaintext ACT, represented by

$$m = (0, 2, 19)^T,$$

the given material gives ciphertext

$$c = (15, 14, 7)^T.$$

The given inverse matrix is

$$E^{-1} \equiv \begin{pmatrix} 8 & 5 & 10 \\ 21 & 8 & 21 \\ 21 & 12 & 8 \end{pmatrix} \pmod{26}.$$

Applying it to $(15, 14, 7)^T$ recovers $(0, 2, 19)^T$.

8.5.4 Known-Plaintext Vulnerability

Note that a 3×3 Hill cipher has nine matrix entries. With enough known plaintext/ciphertext pairs, an attacker can form enough equations to solve for the matrix. This section nevertheless notes that linear/affine ideas can still be useful when combined with non-linear operations.

8.5.5 Caesar with an Increasing Shift

A normal Caesar shift by 3 sends HELLO to KHOOR. This section then gives an increasing-shift version in which the shift changes from one character to the next, producing a different ciphertext for repeated letters. This demonstrates a construction that is neither a single linear nor a single affine map over the whole message.

8.5.6 Optional Enigma Material

The given Unit 8 material includes an optional Enigma discussion: Arthur Scherbius, commercial introduction in the 1920s, later use by Nazi Germany in World War II, and Alan Turing's codebreaking work. The material also points to a video resource.

8.6 Secret Sharing

8.6.1 Two-Point Idea

Use the fact that two points determine a line.

For the four-digit secret 6965, it chooses a prime $p = 10007$ and a random intercept, giving the line

$$y \equiv 6965x + 30 \pmod{10007}.$$

The given shares are

$$(1, 6995), \quad (2, 3953), \quad (3, 911), \quad (4, 7876).$$

Any two shares determine the line and therefore recover the secret intercept 6965.

Step-by-Step Recovering the secret from Alice and Claire Using points $(1, 6995)$ and $(3, 911)$, the slope is computed modulo 10007. Use the inverse

$$2^{-1} \equiv 5004 \pmod{10007}$$

and recovers slope 6965, which opens the lock.

8.6.2 (t, n) Threshold Secret Sharing

Definition In a (t, n) scheme, any t out of n participants can reconstruct the secret.

The previous line example is a $(2, 4)$ scheme. This section then generalizes to schemes such as $(3, 4)$.

8.6.3 Polynomial Secret Sharing

For threshold t , choose a prime p and a random polynomial

$$f(x) = s + a_1x + a_2x^2 + \cdots + a_{t-1}x^{t-1} \pmod{p}.$$

Here s is the secret, all arithmetic is modulo p , and participant i receives

$$(i, f(i)).$$

Any t points determine the polynomial and hence $s = f(0)$.

8.6.4 Recovery as a Linear Algebra Problem

Given t shares, there are t linear equations in t unknown coefficients. The coefficient matrix is a Vandermonde matrix; this section states that it is invertible and that Gaussian elimination or Cramer's rule can then be used.

Example The given $(3, n)$ example uses $p = 101$ and shares

$$(1, 5), (2, 9), (3, 15).$$

The Vandermonde system is

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{pmatrix} \begin{pmatrix} s \\ a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 9 \\ 15 \end{pmatrix} \pmod{101}.$$

This section's Cramer-rule calculation gives

$$s \equiv 89 \times 2^{-1} \pmod{101}.$$

Since

$$2^{-1} \equiv 51 \pmod{101},$$

the secret is

$$s \equiv 95 \pmod{101}.$$

9 Unit 9 — Error-Correcting Codes

9.1 Introduction

9.1.1 (n, k) Binary Codes

A binary (n, k) code has:

- k information bits,
- r redundant/parity bits,
- $n = k + r$ total coded bits.

The code C consists of 2^k codewords in $\mathbb{B}^n$.

Definition An encoding is systematic when the information bits occur directly in the coded output.

9.1.2 Single Parity Check Revisited

Even parity adds one bit so that the total number of 1s is even. The receiver can detect a single-bit error but cannot determine which bit is wrong.

9.1.3 Code Rate

$$R_c = \frac{k}{n}.$$

The effective information rate for a raw link of W bits/s is $R_c W$.

9.1.4 Linear Codes

Definition A code is linear when the encoding function

$$c = f(u)$$

is a linear function. The course considers linear codes because matrix multiplication can represent encoding and decoding efficiently.

9.1.5 Matrix-Matrix Multiplication

If A is $m \times p$ and B is $p \times n$, then AB is $m \times n$. Each entry is the inner product of one row of A and one column of B . This section emphasizes that, in general,

$$AB \neq BA.$$

9.2 Error Detection Capability

9.2.1 Why Single Parity Can Fail

For the $(4, 3)$ even-parity code, a received word can itself be another valid even-parity codeword. When this happens, the decoder sees no parity violation, so the error is undetected.

Definition The Hamming distance $d(x, y)$ is the number of bit positions in which x and y differ.

Definition The Hamming weight $w(x)$ is the number of 1s in x .

Example

$$x = (0, 0, 1, 1, 1), \quad w(x) = 3,$$

$$y = (0, 1, 1, 0, 1), \quad w(y) = 3,$$

and

$$d(x, y) = 2.$$

9.2.2 Hamming Distance is a Metric

This section lists:

1. Non-negativity: $d(x, y) = 0$ iff $x = y$.
2. Symmetry: $d(x, y) = d(y, x)$.
3. Triangle inequality: $d(x, y) \leq d(x, z) + d(z, y)$.

Any function satisfying these is a metric; Euclidean distance in $\mathbb{R}^n$ is given as another example of a metric.

9.2.3 Minimum Distance

Definition

$$d_{\min}$$

is the smallest Hamming distance between any pair of distinct codewords.

Theorem A code is guaranteed to detect up to

$$s = d_{\min} - 1$$

bit errors.

Step-by-Step Why If fewer than $d_{\min}$ errors occur, the received word cannot become another codeword because distinct codewords are at least $d_{\min}$ apart.

Example This section compares:

- (4, 3) even parity: the minimum distance is 2, hence it guarantees detection of at most 1 bit error.
- (5, 1) repetition: $C = \{00000, 11111\}$, so $d_{\min} = 5$, hence it guarantees detection of up to 4 errors.

9.3 Error Correction Capability

Definition Nearest-neighbor decoding chooses a codeword in C having minimum Hamming distance from the received vector y . A tie is broken arbitrarily.

Theorem The guaranteed error-correction capability is

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor.$$

Example A (6, 1) repetition code has $d_{\min} = 6$, so

$$t = \left\lfloor \frac{5}{2} \right\rfloor = 2.$$

9.3.1 (5,1) Repetition Code

For

$$C = \{00000, 11111\},$$

we have

$$d_{\min} = 5, \quad t = 2, \quad R_c = \frac{1}{5}.$$

At 1 Mbps, the effective information rate is 200 kbps.

9.3.2 Repetition with an Extra Parity Bit

Consider an example with

$$u = (u_1, u_2)$$

and

$$c_1 = c_3 = u_1, \quad c_2 = c_4 = u_2, \quad c_5 = u_1 + u_2.$$

The resulting code has four codewords, $d_{\min} = 3$, correction capability $t = 1$, and rate

$$R_c = \frac{2}{5}.$$

At 1 Mbps, the useful information rate is 400 kbps. This section compares it with the (5, 1) repetition code: it is more communication-efficient but has weaker correction capability.

Key Point Higher R_c means better communication efficiency; larger $d_{\min}$ means stronger detection/correction capability.

9.4 Generator Matrices for Encoding

Definition A generator matrix G for a linear (n, k) code is a $k \times n$ binary matrix satisfying

$$c = uG.$$

For a systematic code,

$$G = [I_k \mid P].$$

Example The given (5, 4) even-parity generator matrix is

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix}.$$

Hence

$$c_1 = u_1, \quad c_2 = u_2, \quad c_3 = u_3, \quad c_4 = u_4,$$

and

$$c_5 = u_1 + u_2 + u_3 + u_4.$$

9.4.1 Injectivity of Encoding

This section explains that a linear encoder must be injective so that two different information vectors do not produce the same codeword.

Example The example gives a non-example where two different information vectors map to the same codeword. Such a matrix cannot be used for unambiguous decoding.

9.4.2 Linear Codes as Vector Subspaces

This section optionally views a binary linear code as a vector subspace: it is closed under vector addition (XOR) and scalar multiplication by 0 and 1.

9.4.3 Minimum Distance of a Linear Code

Theorem For a linear code,

$$d_{\min} = \text{minimum weight of a non-zero codeword.}$$

The proof in this section uses linearity and the relation between Hamming distance and binary vector addition.

9.5 Parity-Check Matrices for Decoding

9.5.1 Parity-Check Equation

A parity-check matrix H allows a valid codeword c to satisfy

$$cH^T = 0.$$

For a received vector y , compute the syndrome

$$s = yH^T.$$

If $s = 0$, no error is detected by the parity checks. If $s \neq 0$, at least one parity-check equation has failed.

9.5.2 Simple Parity Example

For the $(5, 4)$ parity code, this section uses

$$H = (1 \ 1 \ 1 \ 1 \ 1),$$

so the check equation is

$$y_1 + y_2 + y_3 + y_4 + y_5 = 0.$$

9.5.3 Systematic Form

For a systematic generator matrix

$$G = [I_k \mid A],$$

the example gives the corresponding parity-check structure

$$H = [A^T \mid I_r].$$

The relationship is binary and the valid codewords satisfy $cH^T = 0$.

9.5.4 Generator Matrix of the (5, 2) Example

The example gives the generator rows

$$G = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}.$$

The corresponding parity-check equations are those used in the syndrome example below.

9.5.5 Syndrome Example

For the given (5, 2) code, the example gives a parity-check matrix corresponding to the three equations

$$y_1 + y_3 = 0,$$

$$y_2 + y_4 = 0,$$

$$y_1 + y_2 + y_5 = 0.$$

For received

$$y = (0, 1, 0, 0, 1),$$

the given syndrome is

$$s = (0, 1, 0),$$

so only the second equation fails. Under the assumption of one bit error, this section identifies the error as the fourth bit.

9.6 More on Hamming Codes

9.6.1 The (7, 4) Hamming Code

This section revisits the same encoding equations as Unit 1:

$$c_1 = u_1, \quad c_2 = u_2, \quad c_3 = u_3, \quad c_4 = u_4,$$

$$c_5 = u_1 + u_2 + u_4,$$

$$c_6 = u_1 + u_3 + u_4,$$

$$c_7 = u_2 + u_3 + u_4.$$

The generator matrix is

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

All 16 codewords generated by the given equations

u	c
0000	0000000
0001	0001111
0010	0010011
0011	0011100
0100	0100101
0101	0101010
0110	0110110
0111	0111001
1000	1000110
1001	1001001
1010	1010101
1011	1011010
1100	1100011
1101	1101100
1110	1110000
1111	1111111

9.6.2 Hamming Code Capability

It follows that the $(7, 4)$ Hamming code has $d_{\min} = 3$. Therefore it can detect up to two-bit errors or correct all single-bit errors. This section emphasizes that these two guarantees cannot be used simultaneously at the same strength: the code can be used either for two-error detection or one-error correction.

9.6.3 Parity-Check Matrix

The given Hamming parity equations are

$$y_1 + y_2 + y_4 + y_5 = 0,$$

$$y_1 + y_3 + y_4 + y_6 = 0,$$

$$y_2 + y_3 + y_4 + y_7 = 0.$$

Thus the example gives

$$H = \begin{pmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

9.6.4 Single-Bit Syndrome Decoding

Step-by-Step

1. Receive y .
2. Compute $s = yH^T$.
3. Identify which parity equations fail.

4. Under the one-error assumption, use the syndrome pattern to locate the erroneous bit.
5. Flip that bit to recover the codeword and then read off u_1, u_2, u_3, u_4 from the systematic positions.

9.6.5 Unit 1 / Test 1 Hamming Example Revisited

For

$$y = (0, 1, 0, 1, 1, 0, 0),$$

the worked example solution uses the three parity checks (shown as a Venn-diagram method in the paper), identifies the first bit as wrong, and gives the recovered message

$$(1, 1, 0, 1).$$

9.6.6 Error Detection versus Correction Trade-off

This section's central quantitative relationships are:

detect up to $d_{\min} - 1$ errors,

correct up to $\left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$ errors.

This should always be read together with the code rate

$$R_c = \frac{k}{n}.$$

More redundancy can increase error-handling capability but reduces the fraction of transmitted bits carrying raw information.

10 Cross-Unit Exam Workflow

10.1 Proving a Function is Injective

Step-by-Step

1. Start with arbitrary x_1, x_2 in the domain.
2. Assume $f(x_1) = f(x_2)$.
3. Simplify until the equation forces $x_1 = x_2$.
4. Conclude injection.

To disprove injection, give one counterexample $x_1 \neq x_2$ with equal outputs.

10.2 Proving a Function is Surjective

Step-by-Step

1. Choose an arbitrary codomain value y .
2. Solve $f(x) = y$ for a domain value x .
3. Check that the resulting x is actually in the domain.
4. Conclude surjection.

To disprove surjection, find one codomain element that is never attained.

10.3 Proving an Equivalence Relation

Check exactly three properties:

1. Reflexive.
2. Symmetric.
3. Transitive.

Common Trap Antisymmetry is **not** one of the three requirements for an equivalence relation.

10.4 Finding an Equivalence Class

Step-by-Step

1. Start from the definition $[a] = \{x : xRa\}$.
2. Substitute the specific relation.
3. Simplify to an equation or description of all related elements.
4. If the problem is geometric, express the class as a curve/line/circle as this section examples do.

10.5 Extended Euclidean Algorithm

Step-by-Step

1. Perform Euclidean division until the remainder is zero.
2. Track coefficients of the original numbers.
3. The final non-zero remainder is the gcd.
4. Read off coefficients x, y satisfying $ax + by = \gcd(a, b)$.

10.6 Finding a Modular Inverse

To find $a^{-1} \pmod{n}$:

1. First check $\gcd(a, n) = 1$.
2. Use extended Euclid to obtain $ax + ny = 1$.
3. Reduce x modulo n .
4. State $a^{-1} \equiv x \pmod{n}$.

10.7 Modular Exponentiation

1. Reduce the base modulo the modulus.
2. Square repeatedly and reduce after every operation.
3. Decompose the exponent into powers of 2.
4. Multiply only the required powers.

10.8 RSA

1. Identify p, q and compute $N = pq$.
2. Compute $\varphi(N) = (p - 1)(q - 1)$.
3. Check $\gcd(e, \varphi(N)) = 1$.
4. Find $d = e^{-1} \pmod{\varphi(N)}$.
5. Encrypt with $m^e \pmod{N}$.
6. Decrypt with $c^d \pmod{N}$.
7. For hand calculations, use square-and-multiply.

10.9 Diffie-Hellman

1. Public: prime p and primitive element α .
2. Alice publishes $\alpha^a \pmod{p}$.
3. Bob publishes $\alpha^b \pmod{p}$.
4. Shared key is either $(\alpha^b)^a$ or $(\alpha^a)^b$ modulo p .
5. Use repeated squaring for numerical calculations.

10.10 CRT

1. Check the moduli are co-prime.
2. Use extended Euclid to obtain Bézout coefficients.
3. Substitute them into the CRT construction.
4. Reduce the result modulo the product of the moduli.

10.11 Error-Correcting Codes

1. Identify (n, k) and compute $R_c = k/n$.
2. Determine or use $d_{\min}$.
3. Detection capability: $d_{\min} - 1$.
4. Correction capability: $\lfloor (d_{\min} - 1)/2 \rfloor$.
5. For a generator matrix, encode using $c = uG$.
6. For a parity-check matrix, compute syndrome $s = yH^T$.

10.12 Exam-Style Coverage

The two given test solutions collectively demonstrate questions involving:

- Vigenère encryption and security comparison with Caesar.
- Function composition, range, surjectivity and inverses.
- Symmetry and antisymmetry proofs/counterexamples.
- Injectivity and surjectivity of floor/modulo functions.
- Equivalence relations and geometric equivalence classes.
- Euler's totient function and prime-factor arguments.
- Divisibility rules proved with modular arithmetic.
- Extended Euclidean algorithm and Bézout coefficients.
- Hamming-code decoding and code-rate comparison.
- CIA triad definitions.
- Relations induced by partitions.
- Logarithmic/exponential function inverses.
- CRT with three congruences.
- Modular inverse via extended Euclid.
- Modular exponentiation / square-and-multiply.
- Order of an element modulo a prime.
- Equivalence relations on $\mathbb{R}^2$.

11 Quick Formula and Definition Reference

11.1 Functions

$$(g \circ f)(x) = g(f(x))$$

$$\text{injective: } f(x_1) = f(x_2) \Rightarrow x_1 = x_2$$

$$\text{surjective: } \text{range}(f) = \text{codomain}(f)$$

11.2 Relations

$$xRy \iff (x, y) \in R$$

$$\text{equivalence} = \text{reflexive} + \text{symmetric} + \text{transitive}$$

$$[a] = \{x : xRa\}$$

11.3 Divisibility and GCD

$$d \mid n \iff \exists k \in \mathbb{Z}, n = dk$$

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

$$\gcd(a, b) = ax + by$$

$$\varphi(p^k) = p^{k-1}(p - 1)$$

$$\varphi(mn) = \varphi(m)\varphi(n) \quad \text{when } \gcd(m, n) = 1$$

11.4 Modular Arithmetic

$$a \equiv b \pmod{n}$$

$$a^{-1} \text{ exists mod } n \iff \gcd(a, n) = 1$$

$$a^{p-1} \equiv 1 \pmod{p} \quad (p \text{ prime, } p \nmid a)$$

$$a^{\varphi(n)} \equiv 1 \pmod{n} \quad (\gcd(a, n) = 1)$$

11.5 RSA

$$N = pq, \quad \varphi(N) = (p - 1)(q - 1)$$

$$ed \equiv 1 \pmod{\varphi(N)}$$

$$c \equiv m^e \pmod{N}, \quad m \equiv c^d \pmod{N}$$

11.6 Diffie-Hellman

$$\begin{aligned}y_A &= \alpha^a \pmod{p}, & y_B &= \alpha^b \pmod{p} \\k &= \alpha^{ab} \pmod{p}\end{aligned}$$

11.7 El Gamal

$$\begin{aligned}e &= \alpha^d \pmod{p} \\c_1 &= \alpha^k \pmod{p}, & c_2 &= e^k m \pmod{p} \\m &= c_2 (c_1^d)^{-1} \pmod{p}\end{aligned}$$

11.8 Linear Algebra

$$\begin{aligned}f(\alpha x + \beta y) &= \alpha f(x) + \beta f(y) \\f(x) &= Ax \\ \det \begin{pmatrix} a & b \\ c & d \end{pmatrix} &= ad - bc \\ A^{-1} &= \frac{1}{\det A} \operatorname{adj}(A)\end{aligned}$$

11.9 Error-Correcting Codes

$$\begin{aligned}R_c &= \frac{k}{n} \\d(x, y) &= \text{number of differing bits} \\d_{\min} &= \text{smallest distance between distinct codewords} \\&\text{detect up to } d_{\min} - 1 \\&\text{correct up to } \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor \\c &= uG, & s &= yH^T\end{aligned}$$

12 Source Coverage Map

The note is organized according to the given course materials. The following map records the given Unit structure used in this document:

Unit	Major topic	Main given subtopics represented in this note
0	Course Overview	CILO, topics, assessment, TBL, important dates
1	Introduction to Coding and Cryptography	communication systems, crypto goals, classical ciphers, modulo arithmetic, error detection/correction
2	Functions	sets, functions, composition, injections/surjections, inverse, countability, hash functions
3	Relations	relation definitions, properties, equivalence, equivalence classes, applications/examples
4	Divisibility and Euclidean Algorithm	divisibility, primes, co-primes, φ , Euclid, extended Euclid, fundamental theorem
5	Modular Arithmetic and CRT	congruence, fields, Diophantine equations, inverses, exponentiation, Fermat, Euler, CRT, Sage-Math
6	Public Key Cryptography	ethics, public-key ideas, signatures, RSA, Diffie-Hellman, El Gamal
7	Digital Certificate	identity problem, authority, certificates, X.509, self-signed certificates, OpenSSL, TBL
8	Linear Algebra for Cryptography	vectors, linearity, matrices, determinants, Hill cipher, secret sharing, Enigma supplement
9	Error-Correcting Codes	code parameters, distance, detection, correction, generator matrices, parity-check matrices, Hamming codes

End of Notes

The mathematical notation, worked examples, terminology, and procedures above are synthesized from the given course documents and the given test materials/solutions.